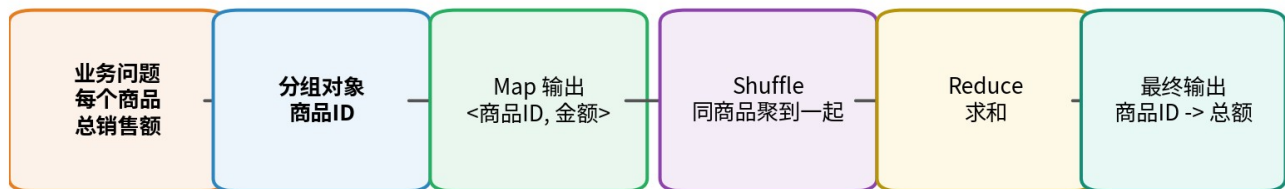


Hadoop MapReduce 原理详解

第九讲：现实任务建模

Key/Value 设计 · 去重 · Top-N · Join · 多阶段 Job

MapReduce 问题建模：先找“按什么分组”，再写代码



核心问题不是“Mapper 怎么写”，而是：哪一个字段应该成为 Key？Value 里必须保留什么信息？

本讲目标：看到一个真实数据任务，先能“建模”，再考虑 Mapper/Reducer 代码。

现实任务建模：Key/Value、去重、Top-N、Join 与多阶段 Job

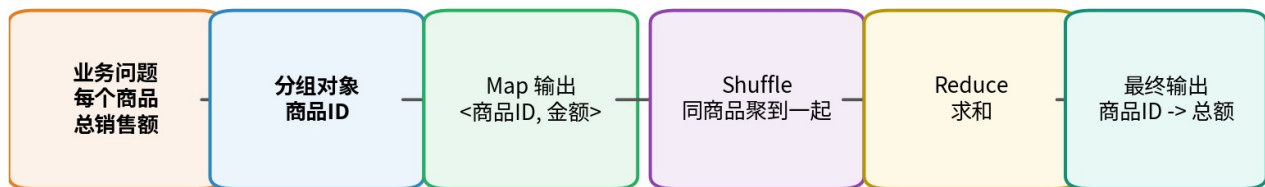
一、本讲学习目标

- 建立一套固定的 MapReduce 问题建模流程，而不是遇到题目就从 Java 语法开始想。
- 能根据“希望哪些记录最终汇聚到一起”来设计 Map 输出 Key。
- 理解 Value 的职责：保留 Reduce 阶段完成计算所必需的信息。
- 掌握计数、求和、平均、去重、Top-N、Join 六类高频业务模式。
- 知道什么时候一个 Job 就够，什么时候必须拆成两个或更多 Job。
- 能判断一个任务是否根本不需要 Reduce，避免为了“像 MapReduce”而强行使用 Reducer。

本讲最重要的一句话：MapReduce 建模的核心，不是先问“map() 里写什么”，而是先问：哪些数据必须在计算后期见面？决定它们见面的字段，就是候选 Key。

二、从业务问题到 Key/Value：固定的六步法

MapReduce 问题建模：先找“按什么分组”，再写代码



核心问题不是“Mapper 怎么写”，而是：哪一个字段应该成为 Key？Value 里必须保留什么信息？

1. 明确最终要输出什么。例如：每个商品总销售额、每个班平均分、访问量最高的 10 个 URL。
2. 找“分组对象”。问自己：哪些记录必须放到一起才能得到最终答案？
3. 把这个分组对象设计成 Map 输出 Key。
4. 设计 Map 输出 Value：Reduce 还缺什么信息，就把什么带过去。
5. 确定 Reduce 对同一个 Key 的 values 做什么：计数、求和、最大值、拼接、关联等。
6. 最后才考虑优化：Combiner、多个 Reducer、自定义 Partitioner、压缩、多 Job。

快速自检：如果你说不清“为什么这个字段是 Key”，通常说明任务还没有建模清楚。

三、六类最常见的 MapReduce 业务模式

常见业务问题 -> MapReduce 模式



看到新任务时，先把它归类成“计数/聚合/去重/排序/连接/多阶段”中的一种或组合。

问题类型	典型 Map 输出	Reduce 做什么	现实例子
计数	<key, 1>	sum(values)	URL PV、错误码次数、商品销量
求和	<key, amount>	sum(amounts)	销售额、流量、时长
平均	<key, (sum,count)>	sum(sum)/sum(count)	平均成绩、平均响应时间
去重	<object, NullWritable>	每个 key 输出一次	用户去重、设备 ID 去重
Top-N	局部候选	再选最大 N 个	热门商品、热门页面
Join	<joinKey, taggedRecord>	按 joinKey 关联	用户信息 + 订单、商品 + 销售

四、案例 1：网站日志 - 统计每个 URL 的访问次数

假设日志经过解析后，每条记录至少有 userId、URL、时间、状态码。现在要求“统计每个 URL 的 PV”。

输入样例

原始记录：			
U001	/index	10:01	200
U002	/product/1	10:01	200
U003	/index	10:02	200
U004	/index	10:03	404

建模时先问：哪些记录必须最终见面？答案是“访问同一个 URL 的记录”。因此 URL 是自然 Key。

现实任务建模：Key/Value、去重、Top-N、Join 与多阶段 Job

MapReduce 数据流

```

Map:
/index      -> 1
/product/1  -> 1
/index      -> 1
/index      -> 1

Shuffle:
/index      -> [1,1,1]
/product/1  -> [1]

Reduce:
/index      -> 3
/product/1  -> 1
  
```

为什么 Value 是 1：因为最终只需要“次数”。每条访问记录贡献一次，所以记录本身的其他字段在这个任务中都可以丢掉。

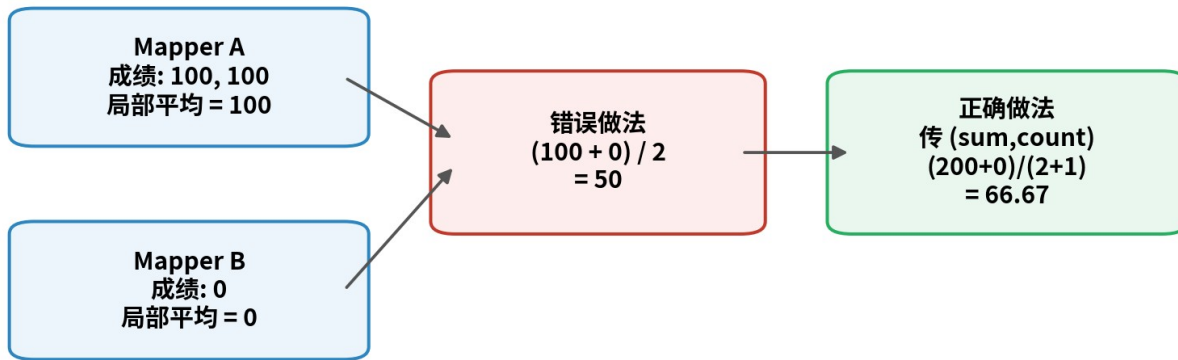
同一份日志只要换一个问题，Key 就可能完全变化：

业务问题	Map Key	Map Value
每个 URL PV	URL	1
每个状态码数量	statusCode	1
每个用户访问次数	userId	1
每小时访问量	hour	1
每个 URL 的独立用户数 UV	(URL,userId) 或多阶段设计	1 / 用户标记

五、案例 2：学生成绩 - 平均值为什么比求和更容易出错

任务：统计每个班级的平均成绩。最直观的 Key 是 classId。但 Value 不能只考虑“当前这一条记录”，还要考虑后面如何安全地合并。

平均值为什么不能“先平均再平均”



可合并的中间状态必须保留足够信息。平均值的“充分状态”是 **sum + count**，而不是 **avg** 本身。

正确的中间状态设计

```
Map:
classA -> (90, 1)
classA -> (80, 1)
classB -> (75, 1)

Reduce(classA):
sum = 90 + 80 = 170
count = 1 + 1 = 2
avg = 170 / 2 = 85
```

底层原则：如果要做局部聚合或 Combiner，中间状态必须能“再次合并”。sum、count 可以相加；average 本身通常不能直接再平均。

这个原则以后会反复出现：标准差、方差、平均速度等指标也不能随便“先算局部结果再平均”，必须设计可组合的统计状态。

六、案例 3：销售数据 - 复合 Key 什么时候出现

原始销售记录可能包含：日期、城市、商品、数量、销售额。不同业务问题会产生不同 Key。

问题	合理的 Map Key	Value	Reduce
每个商品总销售额	productId	amount	求和
每个城市总销售额	city	amount	求和
每天总销售额	date	amount	求和
每个城市每天销售额	(city,date)	amount	求和
每个商品每天销量	(productId,date)	quantity	求和

复合 Key 的本质：当你的“分组条件”由多个字段共同决定时，就需要复合 Key。它不是高级语法，而是业务分组条件的直接编码。

七、案例 4：去重 DISTINCT - 有时 Reduce 几乎什么也不算

任务：海量设备日志中，一个 deviceId 可能出现几千次，现在只想得到“出现过哪些设备”。

按 Key 天然去重

```
Map:
A100 -> Null
A100 -> Null
B205 -> Null
A100 -> Null

Shuffle:
A100 -> [Null, Null, Null]
B205 -> [Null]

Reduce:
A100
B205
```

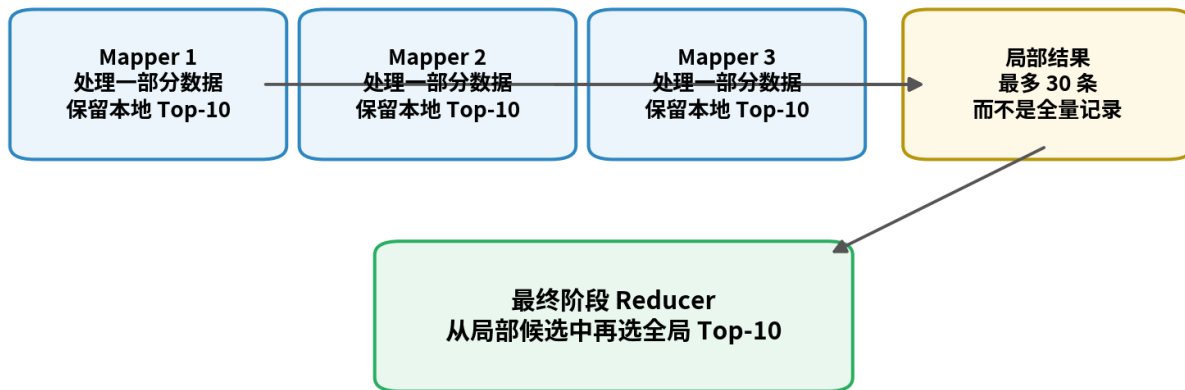
这里 Reduce 的价值不是复杂计算，而是利用 Shuffle 的“相同 Key 聚集”性质。只要每个 Key 输出一次，就完成了 DISTINCT。

不要滥用 Reduce：如果任务只是逐条解析、过滤、格式转换，而且各记录之间不需要见面，那么更合适的是 Map-only Job。

八、案例 5：Top-N - 为什么经常需要“局部 Top-N + 全局 Top-N”

任务：从几十亿条商品访问记录中找访问量最高的 10 个商品。通常不是直接把所有记录交给一个 Reducer 排序，而是分阶段缩小数据。

Top-N：先局部筛选，再做全局 Top-N



本质：尽量在数据靠近 Mapper 的地方先“缩小候选集”，减少 Shuffle 和最终比较量。

一种典型思路：

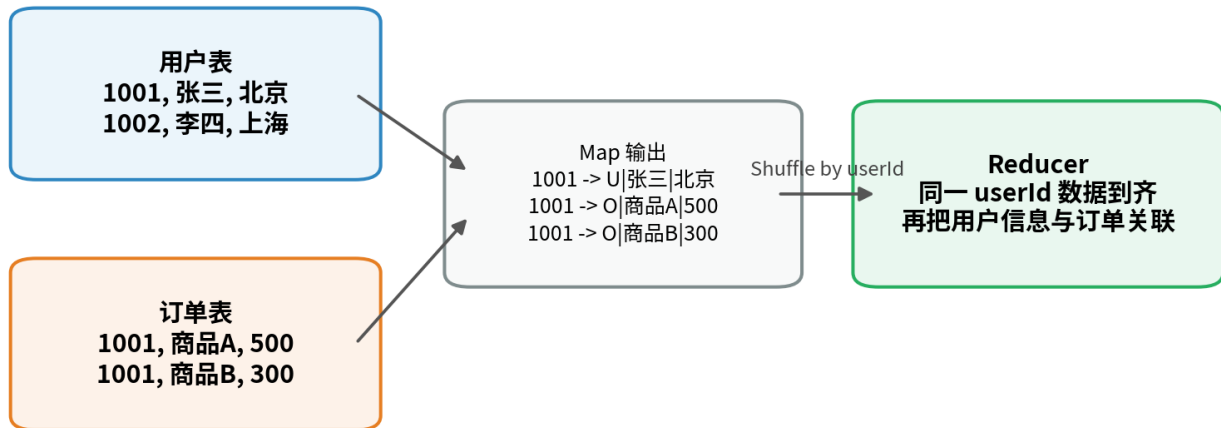
- Job 1：先按商品统计访问次数，得到 productId -> count。
- Job 2：每个 Mapper 在自己读到的一批 product-count 中维护一个大小为 10 的小顶堆，只输出局部 Top-10。
- 最终 Reducer 只需要从所有“局部候选”里再次选 Top-10。

关键思想：Top-N 的优化不是 MapReduce 独有，而是一个普遍算法思想：尽量早地丢弃不可能进入最终答案的数据。

九、案例 6：Join - 两张大表怎样在 MapReduce 里相遇

假设有用户表和订单表，希望输出“用户姓名 + 订单商品 + 金额”。两张表共同字段是 userId，所以 userId 是连接 Key。

Reduce-side Join：把“连接字段”作为 Shuffle Key



代价：两张大表都要参与 Shuffle。若一张表很小，可考虑 Map-side Join / DistributedCache。

Mapper 还必须给 Value 加一个“来源标签”，否则 Reducer 收到多个值时，不知道哪条来自用户表、哪条来自订单表。

Reduce-side Join

用户表 Mapper:
1001 -> U|张三|北京

订单表 Mapper:
1001 -> O|商品 A|500
1001 -> O|商品 B|300

Reducer(1001):
先找到 U 记录
再把它与所有 O 记录组合
输出: 张三, 商品 A, 500
张三, 商品 B, 300

代价：Reduce-side Join 很通用，但两张大表的数据都要经历 Shuffle。若一张表非常小，可以把小表分发到各 Mapper，本地直接查表，这就是 Map-side Join / Broadcast Join 的基本思想。

十、一个 Job 什么时候不够？

什么时候一个 Job 不够？



判断标准：后一步必须依赖“前一步的全局结果”，而不是单条记录即可决定。

多 Job 的本质是构造一个计算 DAG。Spark 后来把这种多阶段计算做得更自然。

MapReduce 一个 Job 的结构天然“一次 Map -> Shuffle -> Reduce”。如果后一个计算依赖前一个计算的全局输出，常常必须再开一个 Job。

任务	一个 Job 是否通常足够	原因
每个商品销售额	是	一次按商品聚合即可
每个班平均分	是	一次按班级聚合即可
所有商品销量 Top-10	通常需要两阶段	先聚合每个商品，再在聚合结果中选 Top-N
先按用户汇总，再按城市统计用户总消费	通常多阶段	第二阶段依赖第一阶段用户级结果
复杂图算法/迭代算法	常常需要很多 Job	每轮结果成为下一轮输入

迁移到 Spark：Spark 的 DAG 本质上就是把“多个相互依赖的计算阶段”统一组织起来，减少手工串联多个 MapReduce Job 的笨重感。

十一、什么时候根本不需要 Reducer？

判断标准非常简单：每条输入记录是否可以在完全不知道其他记录的情况下独立得到最终输出？如果可以，通常可以考虑 Map-only。

任务	是否需要 Reduce	原因
JSON 转 CSV	通常不需要	逐条转换
过滤状态码 != 200 的日志	通常不需要	逐条判断
脱敏手机号/身份证	通常不需要	逐条转换
每个 URL 访问次数	需要	必须汇聚相同 URL
全局去重	需要或等价 Shuffle	重复记录必须相遇
Join 两张大表	通常需要	相同连接键数据必须相遇

现实任务建模：Key/Value、去重、Top-N、Join 与多阶段 Job

十二、Reducer 数量应该怎么想

Reducer 数量不是“业务 Key 有多少就开多少个”。一个 Reducer 可以连续处理很多不同 Key。数量主要是并行度与资源权衡。

- Reducer 太少：每个 Reducer 数据太多，计算时间长，容易形成瓶颈。
- Reducer 太多：Container 启动、Shuffle 连接、输出小文件都会增加开销。
- 若只需要一个全局结果（例如最终全局 Top-10），最后一阶段常设置 1 个 Reducer，但要确保输入已经充分缩小。
- 如果存在热点 Key，仅仅增加 Reducer 可能无效，因为一个逻辑 Key 仍必须进入同一个 Reducer。

先正确，再优化：建模阶段先保证同一逻辑 Key 的数据能正确相遇；性能阶段再考虑 Partitioner、加盐、Combiner、压缩和并行度。

十三、一张“看到题目后怎么做”的决策表

你看到的需求	先问什么	常见 Map Key	常见结构
统计次数	按什么维度数？	维度字段	<key,1> -> sum
求总量	按什么维度累加？	维度字段	<key,value> -> sum
算平均	需要保留哪些可合并状态？	维度字段	<key,(sum,count)>
去重	什么定义为“同一个对象”？	唯一标识	key 唯一化
Top-N	先在哪一级聚合？	通常先按实体聚合	多阶段
Join	两张表靠哪个字段关联？	joinKey	带来源标签的 Value
排序	按哪个字段排序？是否要求全局？	排序字段/复合 Key	Sort / TotalOrder
逐条清洗	记录之间需要相遇吗？	通常无中间聚合 Key	Map-only

十四、把业务语言翻译成 MapReduce 语言：三个练习

练习 1：某视频网站有 userId、videoId、watchSeconds。求每个视频总观看时长。

参考建模：Key = videoId；Value = watchSeconds；Reduce = sum。

练习 2：求每个用户看过多少个“不同的视频”。

参考建模：不能简单 <userId,1>。需要先消除 (userId,videoId) 重复，再按 userId 计数；常见做法是两阶段 Job，或设计能够先按复合 Key 去重的流程。

练习 3：求每个城市消费金额最高的 5 个用户。

参考建模：通常先按 userId 聚合总消费，同时保留 city；再按 city 分组，在每个城市内部选 Top-5。这天然是多阶段问题。

十五、一个通用的 MapReduce 建模模板

以后遇到任何新题目，都先填写这 8 个问题

1. 最终输出是什么？ 例如：<商品 ID，总销售额>
2. 哪些输入记录必须见面？ 同一商品的所有订单。
3. Map Key 是什么？ 商品 ID。
4. Map Value 必须保留什么？ 订单金额。
5. Reduce 如何从 values 得到答案？ sum(values)。
6. 能否 Combiner？ 求和可以。
7. 是否需要多个 Job？ 若只求商品总销售额，不需要；若还要求全局 Top-10，需要下一阶段。
8. 有无倾斜风险？ 检查是否存在超级热门商品。

十六、与前八讲连接起来：你现在真正掌握的是什么

前八讲主要回答“MapReduce 系统内部怎么运行”；这一讲开始回答“我怎样把现实任务交给这个系统”。两部分必须合起来。

层次	你已经学到的内容
业务建模	确定 Key/Value、Reduce 逻辑、是否多阶段
输入阶段	InputFormat、InputSplit、RecordReader
Map 端	Mapper、缓冲区、Spill、Partition、Sort
Shuffle/Reduce	Fetch、Merge、Group、reduce()
资源调度	YARN、Container、ApplicationMaster
可靠性/性能	重试、推测执行、数据倾斜、压缩
高级数据组织	自定义 Partitioner、Secondary Sort、全局排序
编程实现	Driver、Mapper、Reducer、Writable、Context

最终心智模型： MapReduce 的真正能力可以概括为：把海量记录分散处理，再通过 Key 让“应该见面的数据”在 Shuffle 后见面，最后完成聚合、关联或排序。

十七、本讲常见疑问

Q1: 是不是所有 MapReduce 题目都应该有 Reduce?

不是。逐条清洗、转换、过滤往往使用 Map-only。

Q2: Key 是不是一定来自原始数据中的某一列?

不一定。Key 可以由多个字段组合，也可以由程序计算得到，例如 (city,date)、时间窗口、哈希桶。

Q3: Value 是不是越多越保险?

不是。Value 越大，Shuffle 成本越高。只保留后续计算真正需要的信息。

Q4: 为什么 Join 要给 Value 加 U/O 标签?

因为同一个 Key 下会混合两张表的数据，Reducer 必须识别来源。

Q5: Top-N 为什么不能直接一个 Reducer 全部排序?

能，但规模大时会形成单点瓶颈。局部筛选可以大幅缩小最终阶段输入。

Q6: 多 Job 是不是说明设计不好?

不是。很多问题天然包含多个依赖阶段。MapReduce 只是要求把阶段显式拆开。

Q7: 平均值为什么不能作为 Combiner 结果?

普通平均值不可直接合并；应保留 sum 和 count，再在最终阶段计算 avg。

Q8: 多个 Reducer 会不会导致同一个 Key 被拆开?

正常 Partitioner 下不会。同一个 Key 必须映射到同一个分区，否则结果会错误。

Q9: Reducer 的数量和 Key 数量有什么关系?

没有一一对应关系。一个 Reducer 会处理很多 Key。

Q10: MapReduce 现在还值得学这些建模吗?

值得。Spark/Flink/分布式 SQL 依然大量使用分区、Shuffle、聚合、Join、数据倾斜等同一类思想。

十八、本讲总结

- 业务问题先转化为“谁应该和谁见面”，再决定 Key。
- Value 只保留后续计算必需的信息，越精简越能降低 Shuffle 成本。
- 计数、求和、平均、去重、Top-N、Join 是最值得先掌握的六类模式。
- 平均值等统计量要设计“可合并的中间状态”，不能只传最终形式。
- 全局 Top-N、先聚合后再分组等问题通常需要多个 Job。
- 逐条独立处理的任务可以使用 Map-only，不应强行引入 Reduce。
- MapReduce 建模能力的核心，是把现实业务语言翻译成 Key/Value 与数据依赖。

常见业务问题 -> MapReduce 模式



看到新任务时，先把它归类成“计数/聚合/去重/排序/连接/多阶段”中的一种或组合。

下一讲预告：下一讲适合做一次“完整实战”：从一份真实日志/销售数据开始，自己设计 Mapper、Reducer、Driver，在 Hadoop 3.3.5 环境中编译、运行、查看 YARN 日志和输出，并分析 Counters。

参考资料

Apache Hadoop MapReduce API 文档：Mapper、Reducer 等类的官方语义。本文教学中的 Map/Reduce 输入输出关系、Shuffle/Sort/Reduce 阶段与 Secondary Sort 相关描述均与官方 API 语义保持一致。

<https://hadoop.apache.org/docs/stable/api/org/apache/hadoop/mapreduce/Mapper.html>

<https://hadoop.apache.org/docs/stable/api/org/apache/hadoop/mapreduce/Reducer.html>

现实任务建模：Key/Value、去重、Top-N、Join 与多阶段 Job